

9장 최적화 알고리즘(Optimization Algorithms)

written by Han Suebin

9.1 제약조건이 없는 최적화

9.1.1 정의

제약조건이 없는 최적화는 목적 함수 $f(\mathbf{x})$ 의 지형만을 보고 가장 낮은 곳을 찾는 과정입니다.

1. 최소점의 종류와 수치 예시

- 전역 최소점 (Global): 모든 영역 통틀어 최저점.
 - 예: $f(x) = (x - 3)^2 + 5$ 의 전역 최소점은 $x = 3$ (이때 함수값 5).
- 극소점 (Local): 특정 근방(ϵ) 내에서의 최저점.
 - 주변보다 낮기만 하면 되며, 여러 개 존재할 수 있습니다.
- 엄격한 극소점 (Strict): 주변 점들보다 함수값이 작음(<)을 만족.
 - 바닥이 평평하지 않고 뽕족하거나 둥근 골짜기를 의미합니다.

2. 임계점(Critical Point)의 역할

- 정의: $\nabla f(\bar{\mathbf{x}}) = \mathbf{0}$ 인 지점.
- 의미: 지형의 기울기가 평평해지는 지점으로 최적해를 찾기 위한 첫 번째 후보지가 됩니다.

3. 미분이 불가능한 경우 (Exception)

함수가 매끄럽지 않은 경우(예: $f(x) = |x|$), 미분값이 0이 아니거나 존재하지 않더라도 극소점이 될 수 있습니다.

- 이런 뽕족한 지점을 다루기 위해 '서브그래디언트(Subgradient)' 같은 개념을 사용하기도 합니다.

최적화 알고리즘의 목표는 대개 임계점 중에서 전역 최소점을 찾아내는 것입니다.

9.1.2 제약조건이 없는 최적화 문제의 최적화 조건 (Optimality Conditions for Unconstrained Optimization)

최적화 알고리즘이 특정 지점 $\bar{x}$ 에 도달했을 때, 그곳이 정말 전역 최소/최대인지 판별하는 기준입니다.

1. 하강 방향 (Descent Direction): 어디로 가야 낮아질까?

현재 위치에서 함수값을 낮출 수 있는 방향 벡터 d 를 찾는 기준입니다.

- 수식: $\nabla f(\bar{x})^\top d < 0$ (기울기와 반대 방향으로 이동할 때 성립)
- 예제: $f(x_1, x_2) = x_1^2 + 2x_2^2$, 현재 위치 $\bar{x} = [1, 1]^\top$
 - 그래디언트 계산: $\nabla f = \begin{bmatrix} 2x_1 \\ 4x_2 \end{bmatrix} \rightarrow \nabla f(1, 1) = \begin{bmatrix} 2 \\ 4 \end{bmatrix}$
 - 방향 벡터 설정: $d = \begin{bmatrix} -1 \\ -1 \end{bmatrix}$ (원점 방향으로 이동)
 - 계산: $\nabla f^\top d = [2 \ 4] \begin{bmatrix} -1 \\ -1 \end{bmatrix} = -2 - 4 = -6 < 0$
 - 결론: 내적값이 음수이므로, 이 방향은 함수값을 확실히 낮추는 **하강 방향**입니다.

2. 일차 및 이차 필요조건 (Necessary Conditions)

극소점이 되기 위한 최소한의 자격 요건입니다.

- 일차 필요조건: 기울기가 평평해야 함 ($\nabla f = 0$)
- 이차 필요조건: 지형이 아래로 볼록하거나 최소한 평평해야 함 ($\nabla^2 f \succeq 0$, 양의 준정부호)
- 예제: $f(x) = x^3$ (반례 확인)
 - $x = 0$ 에서 $f'(0) = 3(0)^2 = 0$ (일차 필요조건 만족)
 - $x = 0$ 에서 $f''(0) = 6(0) = 0$ (이차 필요조건 만족, $0 \geq 0$)
 - 판단: 필요조건은 모두 만족하지만, $x = 0$ 좌우에서 함수값이 증감을 반복하므로 **극소점이 아닙니다**. (필요조건만으로는 부족함을 보여줌)

3. 최적화 충분조건 (Sufficient Conditions)

이 조건까지 만족하면 **확실하게** 엄격한 극소점이라고 판정하는 기준입니다.

- 조건: $\nabla f = 0$ 이면서 동시에 $\nabla^2 f \succ 0$ (**양의 정부호**)일 것.
- 예제: $f(x_1, x_2) = x_1^2 + x_1x_2 + x_2^2$
 - 일차 조건: $\nabla f = \begin{bmatrix} 2x_1 + x_2 \\ x_1 + 2x_2 \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \end{bmatrix} \implies \bar{x} = [0, 0]^\top$
 - 이차 조건 (헤시안): $\nabla^2 f = \begin{bmatrix} 2 & 1 \\ 1 & 2 \end{bmatrix}$
 - 고유값 확인: $\det(A - \lambda I) = (2 - \lambda)^2 - 1 = 0 \rightarrow \lambda_1 = 3, \lambda_2 = 1$
 - 판단: 고유값이 모두 양수이므로 헤시안이 양의 정부호입니다. 따라서 $(0, 0)$ 은 엄격한 극소점입니다.

다.

4. 안장점 (Saddle Point)

기울기는 0이지만, 방향에 따라 오르막과 내리막이 공존하는 지점입니다.

- 예제: $f(x_1, x_2) = x_1^2 - x_2^2$
 - 임계점: $(0, 0)$ 에서 $\nabla f = \mathbf{0}$
 - 헤시안: $\nabla^2 f = \begin{bmatrix} 2 & 0 \\ 0 & -2 \end{bmatrix}$
 - 고유값 확인: $\lambda_1 = 2, \lambda_2 = -2$ (양수와 음수가 섞여 있음)
 - 판단: x_1 방향으로 아래로 볼록($\cup$)하지만 x_2 방향으로 위로 볼록($\cap$)입니다. 따라서 이곳은 **안장점**입니다.

9.2 선형탐색법(Line Search Methods) [^1]

9.2.1 선형탐색법(Line Search Methods)

현재 위치($\mathbf{x}_k$)에서 새로운 위치($\mathbf{x}_{k+1}$)로 이동하는 반복적인 과정입니다.

1. 탐색 방향 ($\mathbf{d}_k$)

단순히 '내리막'으로 가는 것과 '최저점을 정확히 조준해서' 가는 것은 효율성에서 큰 차이가 납니다.

- 예제: $f(x_1, x_2) = x_1^2 + 10x_2^2$ (x_2 축 방향 경사가 10배 급한 타원형 골짜기)
- 현재 위치: $\mathbf{x}_k = \begin{bmatrix} 1 \\ 1 \end{bmatrix}$, 기울기: $\nabla f_k = \begin{bmatrix} 2 \\ 20 \end{bmatrix}$

① 경사하강법 (Gradient Descent): $B_k = I$

- 계산: $\mathbf{d}_k = -I \begin{bmatrix} 2 \\ 20 \end{bmatrix} = \begin{bmatrix} -2 \\ -20 \end{bmatrix}$
- 문제점: 현재 지점에서 가장 가파른 방향(x_2 축 쪽)으로만 쏠립니다. 정답인 $(0, 0)$ 을 향해 대각선으로 가는 게 아니라, 골짜기 옆벽을 타고 지그재그로 내려가게 되어 매우 느립니다.

② 뉴턴 방법 (Newton's Method): $B_k = \nabla^2 f$ (Hessian)

- 계산: $\nabla^2 f = \begin{bmatrix} 2 & 0 \\ 0 & 20 \end{bmatrix} \implies B_k^{-1} = \begin{bmatrix} 0.5 & 0 \\ 0 & 0.05 \end{bmatrix}$
- 방향: $\mathbf{d}_k = - \begin{bmatrix} 0.5 & 0 \\ 0 & 0.05 \end{bmatrix} \begin{bmatrix} 2 \\ 20 \end{bmatrix} = \begin{bmatrix} -1 \\ -1 \end{bmatrix}$
- 지형의 휘어진 정도(Hessian)를 미리 파악하여 방향을 보정합니다. 현재 위치 $(1, 1)$ 에서 정확히 원점 $(0, 0)$ 을 직선으로 조준하게 됩니다.

2. 스텝 사이즈 (η_k)

방향을 잘 정했어도 보폭이 적절하지 않으면 목표 지점을 지나치거나 도착하지 못합니다.

- 고정 스텝 ($\eta_k = 0.1$):

- $\mathbf{x}_{k+1} = \begin{bmatrix} 1 \\ 1 \end{bmatrix} + 0.1 \begin{bmatrix} -1 \\ -1 \end{bmatrix} = \begin{bmatrix} 0.9 \\ 0.9 \end{bmatrix}$

- 아무리 방향이 좋아도 보폭이 작으면 수만 번의 걸음이 필요합니다.

- 최적 선형 탐색 (Exact Line Search): 이 방향에서 가장 낮은 곳까지 한 번에 가자!

- $f(1 - \eta, 1 - \eta) = 11(1 - \eta)^2$ 가 최소가 되는 $\eta = 1.0$ 을 계산해냅니다.

- $\mathbf{x}_{k+1} = \begin{bmatrix} 1 \\ 1 \end{bmatrix} + 1.0 \begin{bmatrix} -1 \\ -1 \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \end{bmatrix}$ (단 한 걸음에 정답 도달)

- 불완전 선형 탐색 (Inexact Line Search):

- 실제로는 η 를 정확히 계산하는 게 쉽지 않습니다. 그래서 지금보다 확실히 낮아지기만 하면 일단 멈추자라는 **Armijo 조건** 등을 사용하기도 합니다.

3. 하강 방향의 보장

우리가 정한 방향($\mathbf{d}_k$)이 정말 내려가는 길인지 확인하는 수학적 방법입니다.

- 조건: 기울기(∇f_k)와 방향($\mathbf{d}_k$)의 내적이 음수(< 0)여야 함.

- 수학적 안전장치 ($B_k \succ 0$):

- B_k 가 양의 정부호(Positive Definite)라면, $\mathbf{d}_k^T \nabla f_k = -\nabla f_k^T B_k^{-1} \nabla f_k$ 는 무조건 음수가 됩니다.

- 확인:

- 위 예제에서 $\mathbf{d}_k^T \nabla f_k = \begin{bmatrix} -1 & -1 \end{bmatrix} \begin{bmatrix} 2 \\ 20 \end{bmatrix} = -22$

- 이 마이너스 값이 하강 방향을 보장합니다.

9.2.2 학습률 (Learning Rate)

최적화 알고리즘이 좋은 방향($\mathbf{d}_k$)을 찾았더라도, 얼마나 멀리 갈지(η_k)를 잘못 정하면 최적해에 도달할 수 없습니다. **Wolfe 조건**은 이 정도 보폭이면 충분히 합리적이라고 제시해줍니다.

Wolfe 조건 (Wolfe Conditions)

현재 지점에서 정해진 방향으로만 움직일 수 있는 일차원 함수 $\phi(\eta) = f(\mathbf{x}_k + \eta \mathbf{d}_k)$ 를 가정해 봅시다.

[예제]

- 함수: $f(x) = x^2$ (아래로 볼록한 골짜기)
- 현재 위치: $x_k = 2$ ($f(2) = 4$)
- 현재 기울기: $f'(2) = 4$

- 탐색 방향: $d_k = -1$ (왼쪽 내리막 방향)
- 방향 미분값: $\nabla f_k^T d_k = 4 \times (-1) = -4$ (현재 내리막 경사가 -4)

조건 (i): Armijo 조건 (충분한 감소 조건)

"보폭이 너무 커서 골짜기 건너편 오르막까지 치고 올라가지 마라!"

- 수식: $f(\mathbf{x}_{new}) \leq f(\mathbf{x}_k) + c_1 \eta_k \nabla f_k^T \mathbf{d}_k$
- 의미: 실제 함수값의 감소량이 우리가 예상한 최소한의 감소량(c_1 비율)보다는 커야 합니다. 즉, **보폭의 상한선**을 결정합니다.
- $c_1 = 0.1, \eta = 1$ 일 때:
 - 실제 이동 후 높이: $f(2 - 1) = f(1) = 1$
 - 허용 최대 높이: $4 + (0.1 \times 1 \times -4) = 4 - 0.4 = 3.6$
 - 판단: $1 \leq 3.6$ 이므로 적당히 잘 내려왔습니다. 만약 $\eta = 3$ 이었다면 $f(-1) = 1$ 로 다시 올라갔겠지만, 이 조건이 보폭이 너무 큰 것을 막아줍니다.

조건 (ii): 곡률 조건 (Curvature Condition)

"경사가 여전히 너무 가파르다면 더 갈 여지가 있다."

- 수식: $\nabla f(\mathbf{x}_{new})^T \mathbf{d}_k \geq c_2 \nabla f_k^T \mathbf{d}_k$
- 의미: 이동한 지점의 기울기가 처음보다 완만해져야 합니다. (음수 값이 커져서 0에 가까워져야 함). 즉, **보폭의 하한선**을 결정합니다.
- $c_2 = 0.9, \eta = 0.1$ 일 때:
 - 이동 후 지점: $x = 1.9$
 - 이동 후 기울기: $f'(1.9) = 3.8 \rightarrow$ 방향(-1) 고려 시 **-3.8**
 - 최소 허용 기울기: $0.9 \times (-4) = -3.6$
 - 판단: $-3.8 \geq -3.6$ 은 으로 기울기가 여전히 너무 가파릅니다. 더 멀리 가야 합니다.

강한 Wolfe 조건 (Strong Wolfe Conditions)

기존 Wolfe 조건에 절댓값을 씌워 기울기의 범위를 더 좁힙니다. 단순히 기울기가 완만해지는 것을 넘어, 골짜기 바닥을 지나쳐서 반대편 오르막이 가팔라지는 것까지 방지합니다. 즉, 기울기가 거의 0에 가까운 지점(Stationary Point)에 안착하도록 강제합니다.

보조정리 9.2

"하강 방향이거만 하면, 정답 보폭(η)은 반드시 어딘가에 존재한다."

이 정리는 우리가 아무리 복잡한 함수를 만나더라도, $0 < c_1 < c_2 < 1$ 이라는 조건만 지킨다면 Wolfe 조건을 만족하는 학습률 구간이 **반드시 존재함**을 증명합니다. 최적화 알고리즘이 적절한 보폭을 찾지 못해 영원히 헤매는 일은 없습니다.

9.2.3 수렴 속도(Rate of Convergence)

최적화 알고리즘의 효율성은 오차가 줄어드는 비율에 의해 등급이 나뉩니다. 여기서 "q-"는 **Quotient(몫)**를 의미하며, 앞뒤 오차의 비율을 통해 속도를 정의한다는 의미입니다.

1. q-선형 수렴 (q-linear): 일정한 비율로 감소

경사하강법(Gradient Descent)이 보통 이 속도를 가지며, 매 스텝마다 오차가 일정한 비율(c)로 줄어듭니다.

- 수식: $\|\mathbf{x}_{k+1} - \mathbf{x}_*\| \leq c\|\mathbf{x}_k - \mathbf{x}_*\|$ (단, $0 < c < 1$)
- 예제 ($c = 0.5$ 가정, 오차 100에서 시작):
 - 0단계: 100
 - 1단계: $100 \times 0.5 = 50$
 - 2단계: $50 \times 0.5 = 25$
 - 3단계: $25 \times 0.5 = 12.5$
 - 10단계: $100 \times (0.5)^{10} \approx 0.097$
- 특징: 오차가 일정하게 줄어들긴 하지만, 정답에 가까워질수록 줄어드는 절대량이 작아져 시간이 꽤 걸립니다.

2. q-초선형 수렴 (q-superlinear): 속도에 가속도가 붙음

준뉴턴 방법(Quasi-Newton)이 목표로 하는 속도입니다. 오차를 줄이는 비율(c_k) 자체가 스텝이 지날수록 점점 작아집니다.

- 수식: $\|\mathbf{x}_{k+1} - \mathbf{x}_*\| \leq c_k\|\mathbf{x}_k - \mathbf{x}_*\|$ (단, $c_k \rightarrow 0$)
- 예제 (c_k 가 매번 0.1배씩 작아진다고 가정):
 - 0단계: 100
 - 1단계 ($c_0 = 0.5$): $100 \times 0.5 = 50$
 - 2단계 ($c_1 = 0.05$): $50 \times 0.05 = 2.5$
 - 3단계 ($c_2 = 0.005$): $2.5 \times 0.005 = 0.0125$
- 특징: 처음에는 선형처럼 보이다가 어느 순간 정답으로 속도가 빨라집니다.

3. q-이차 수렴 (q-quadratic): 오차의 제곱에 비례

- 수식: $\|\mathbf{x}_{k+1} - \mathbf{x}_*\| \leq C\|\mathbf{x}_k - \mathbf{x}_*\|^2$ (단, $C = 1$ 가정)
- 수치 예제 (오차가 0.1인 구간부터 시작):
 - 0단계: 0.1 (10^{-1})
 - 1단계: $0.1^2 = 0.01$ (10^{-2})
 - 2단계: $0.01^2 = 0.0001$ (10^{-4})
 - 3단계: $0.0001^2 = 0.00000001$ (10^{-8})
- 특징: 압도적으로 빠릅니다. 단, 초기값이 정답 근처여야 한다는 조건이 붙습니다.

오차 e_k 의 변화 비교

단계 (k)	q-선형 (Linear)	q-초선형 (Superlinear)	q-이차 (Quadratic)
0 (시작)	100	100	0.1
1 step	50 (0.5배)	50 (0.5배)	0.01 (제곱)
2 step	25 (0.5배)	2.5 (0.05배)	0.0001 (제곱)
3 step	12.5 (0.5배)	0.0125 (0.005배)	10^{-8} (제곱)

9.2.4 좌표하강법(Coordinate Descent Methods) 해설

좌표하강법은 전체 변수를 한꺼번에 수정하는 대신, 한 번에 하나의 변수만 최적화하는 기법입니다.

메커니즘

- 변수 $x_1, x_2, \dots, x_n$ 중 하나(x_j)를 선택합니다.
- 나머지 변수는 현재 값으로 고정하고, x_j 에 대해서만 목적함수를 최소화합니다.
- 모든 변수에 대해 이 과정을 순차적으로 반복합니다.

예제: $f(x, y) = (x - 1)^2 + (y - 2)^2$

- 시작점: $(0, 0)$
- Round 1 (x 업데이트): $y = 0$ 고정 $\rightarrow f(x, 0) = (x - 1)^2 + 4$. 최소점은 $x = 1$. 위치: $(1, 0)$
- Round 1 (y 업데이트): $x = 1$ 고정 $\rightarrow f(1, y) = (y - 2)^2$. 최소점은 $y = 2$. 위치: $(1, 2)$
- 결과: 단 1회 순회만으로 최적해 $(1, 2)$ 에 도달.

9.3 최적화 알고리즘(Optimization Algorithms)

9.3.1 준뉴턴 방법(Quasi-Newton methods)

준뉴턴 방법의 기본 개념

준뉴턴 방법(Quasi-Newton)은 2차 미분(Hessian)을 직접 계산하지 않고도 그에 준하는 성능을 내는 알고리즘입니다. 그중 **BFGS**는 현대 최적화에서 표준적으로 사용되는 방식입니다.

준뉴턴 방법의 핵심은 진짜 2차 미분값($\nabla^2 f$)을 매번 구하는 대신, 과거의 데이터(그래디언트 변화량)를 축적하여 Hessian의 근사치(B_k)를 업데이트하는 것입니다.

BFGS 방식

함수 $f(x) = x^2$ 에서 $x_k = 2$ 일 때 x_{k+1} 로 나아가는 과정을 예시로 들어봅시다.

- Step 1: 문제 설정
현재 지점 x_k 에서 함수를 q_k 로 근사합니다.
 - 수치: $x_k = 2, f(2) = 4, \nabla f_k = 4$
 - 초기 근사 Hessian $B_k = 1$ (정보가 없을 때는 보통 단위 행렬로 시작)
- Step 2: 탐색 방향 결정
근사된 Hessian의 역행렬($H_k = B_k^{-1}$)을 이용해 방향을 정합니다.
 - 계산: $d_k = -H_k \nabla f_k = -1 \times 4 = -4$
- Step 3: 선형 탐색 (Line Search)
Wolfe 조건을 만족하는 보폭 η_k 를 찾습니다.
 - 수치: $\eta_k = 0.5$ 라고 가정하면, $x_{k+1} = 2 + 0.5(-4) = 0$
- Step 4: 변화량 측정 (s_k, y_k)
실제로 이동한 거리(s_k)와 그에 따른 기울기 변화(y_k)를 기록합니다.
 - s_k (위치 변화): $0 - 2 = -2$
 - y_k (기울기 변화): $\nabla f(0) - \nabla f(2) = 0 - 4 = -4$
- Step 5: Secant 조건 확인
새로운 근사 Hessian B_{k+1} 은 다음 조건을 반드시 만족해야 합니다.
 - 조건: $B_{k+1} s_k = y_k \implies B_{k+1}(-2) = -4$
 - 해석: 위치를 -2만큼 바꿨을 때 기울기가 -4만큼 변했으니, 이 지점의 곡률은 2 정도 되겠군!
- Step 6: BFGS 업데이트 공식 적용
이전의 H_k 를 s_k 와 y_k 데이터를 사용하여 업데이트합니다.
 - 결과: 업데이트 후 $H_{k+1} \approx 0.5$ (즉, $B_{k+1} \approx 2$)
 - 실제 x^2 의 Hessian 값인 2를 단 한 번의 업데이트로 정확히 찾아냈습니다!
- Step 7: 반복
이제 업데이트 된 곡률 정보(B_{k+1})를 가지고 다음 단계로 이동합니다.

왜 BFGS인가?

특징	경사하강법	뉴턴 방법	준뉴턴 (BFGS)
필요 정보	1차 미분 (Gradient)	2차 미분 (Hessian)	1차 미분 (Gradient)
수렴 속도	선형 (Linear)	이차 (Quadratic)	초선형 (Superlinear)
계산 비용	매우 낮음	매우 높음 ($O(n^3)$)	적절함 ($O(n^2)$)
안정성	높음	초기값에 매우 민감	매우 높음 (정부호 유지)

L-BFGS 방식 (Limited-memory BFGS)

L-BFGS의 핵심은 H_k 라는 거대한 행렬($n \times n$)을 직접 만들지 않는 것입니다. 대신 최근 m 개의 이동 거리(s)와 기울기 변화(y) 벡터만 저장해두었다가, **Two-loop Recursion**이라는 알고리즘으로 탐색 방향을 계산합니다.

[예시]

- 목적 함수: $f(x) = x^2$ (최적해 $x_* = 0$)
- 현재 지점 (x_k): 2 (이때의 기울기 $\nabla f_k = 4$)
- 메모리 설정 ($m = 1$): 바로 직전의 데이터 딱 하나만 기억한다고 가정합니다.
 - $s_{k-1} = -2$ (직전 단계에서 -2만큼 이동함)
 - $y_{k-1} = -4$ (그로 인해 기울기가 -4만큼 변함)
 - $\rho_{k-1} = 1/(y_{k-1}^\top s_{k-1}) = 1/8$

Two-loop Recursion의 수치적 추적

이제 $n \times n$ 행렬 곱셈 없이, 오직 저장된 숫자들로만 다음 방향 d_k 를 찾아봅시다.

- Step 1: Backward Loop
현재의 기울기 $q = \nabla f_k = 4$ 에서 시작하여 과거로 거슬러 올라갑니다.
 1. $\alpha = \rho \cdot s^\top q = (1/8) \cdot (-2) \cdot 4 = -1$
 2. $q = q - \alpha \cdot y = 4 - (-1) \cdot (-4) = 0$
 - 결과: 현재 기울기에서 과거 이동에 의한 곡률 성분을 제거하니 $q = 0$ 이 되었습니다.
- Step 2: Initial Scaling (보폭 단위 조절)
초기 역해시안 H_k^0 를 가장 최근에 측정된 곡률로 가볍게 설정합니다.
 1. $\gamma = \frac{s^\top s}{y^\top s} = \frac{(-2)^2}{(-4 \cdot -2)} = \frac{4}{8} = 0.5$
 2. $r = \gamma \cdot q = 0.5 \cdot 0 = 0$
- Step 3: Forward Loop (현재의 보정치 다시 더하기)
다시 현재 시점으로 돌아오며 방향을 최종 보정합니다.
 1. $\beta = \rho \cdot y^\top r = (1/8) \cdot (-4) \cdot 0 = 0$
 2. $r = r + s \cdot (\alpha - \beta) = 0 + (-2) \cdot (-1 - 0) = 2$
 - 최종 방향 결정: $d_k = -r = -2$

결과 분석

- 최종 업데이트: $x_{k+1} = x_k + \eta d_k = 2 + 0.5(-2) = 1$
- 결론: BFGS처럼 거대한 행렬을 저장하지 않고도, 단 몇 개의 숫자(s, y)만 기억하여 뉴턴 방법과 동일한 방향을 정확히 찾아냈습니다.

9.3.3 적응적 학습률 방법(Adaptive learning rate methods)

고정된 학습률을 사용하는 경사 하강법의 한계

함수 $f(x, y) = 10x^2 + 0.1y^2$ 가 있다고 가정해 봅시다.

- 지형: x 방향은 경사가 매우 가파르고($10x^2$), y 방향은 매우 완만합니다($0.1y^2$).
- 기울기: $\nabla f = [20x, 0.2y]^\top$

[문제 상황] 현재 위치 $(1, 1)$, 고정 학습률 $\eta = 0.2$ 라면?

- x 방향: 업데이트 크기가 $0.2 \times 20 = 4.0$ 입니다. 원래 위치가 1인데 4나 움직이니 최저점(0)을 훌쩍 지나쳐 반대편으로 **진동**합니다.
- y 방향: 업데이트 크기가 $0.2 \times 0.2 = 0.04$ 입니다. 너무 조금 움직여서 목표점에 도착하려면 한참 멀었습니다. (**수렴 지연**)

그래디언트 정규화

위 문제를 해결하기 위해 **기울기가 크면 보폭을 줄이고, 작으면 보폭을 키우자**는 아이디어가 등장합니다.

동일한 함수에서 정규화($\epsilon = 0.1$ 로 설정)를 적용해 봅시다.

- x 방향 기울기 $g_x = 20$, y 방향 기울기 $g_y = 0.2$
- 업데이트 공식: $\Delta x = \frac{\epsilon}{\sqrt{g^2 + \delta}} \cdot g \approx \epsilon \cdot \text{sign}(g)$

$$1. x \text{ 업데이트: } \frac{0.1}{\sqrt{20^2}} \cdot 20 = 0.1$$

$$2. y \text{ 업데이트: } \frac{0.1}{\sqrt{0.2^2}} \cdot 0.2 = 0.1$$

결론: 두 방향 모두 **동일하게 0.1만큼** 움직이게 되었습니다!

하지만 이 방식에는 한계가 존재합니다. 바로 **최적점 근처에서도 보폭을 줄이지 않는다**는 점입니다.

- 상황: 최적점에 거의 다 와서 기울기가 0.00001이 되었다고 칩시다.
- 계산: $\frac{\epsilon}{0.00001} \cdot 0.00001 = \epsilon$
- 결과: 기울기가 아무리 작아져도 보폭은 여전히 ϵ (위 예제에선 0.1)입니다. 목표점에 안착해야 하는데, 계속 0.1씩 왔다 갔다 하며 **반동(Overshooting)**이 생기게 됩니다. 즉, 기울기가 작아졌으니 천천히 움직여야 한다는 정보가 반영되지 못합니다.

이 반동 문제를 해결하기 위해 현대적인 적응적 알고리즘들이 탄생했습니다.

적응적 학습률 방법은 각 변수의 특성에 맞춰 보폭을 자동으로 조절합니다. 각 알고리즘이 수치적으로 어떻게 다른 보폭을 만들어내는지 비교해 보겠습니다. (모든 예제는 기본 학습률 $\eta = 1.0$ 가정)

AdaGrad

핵심 아이디어: "많이 움직인 변수는 보폭을 줄이고($\eta \downarrow$), 드물게 움직인 변수는 보폭을 늘리자($\eta \uparrow$)."

- 수식: $r_k = r_{k-1} + g_k^2 \rightarrow \Delta x = -\frac{\eta}{\sqrt{r_k + \epsilon}} g_k$
- 예제 (매 스텝 기울기 $g = 10$ 이 일정하게 발생하는 경우):
 - 1회차: $r = 10^2 = 100 \Rightarrow \Delta x = -\frac{1}{\sqrt{100}} \cdot 10 = -1.0$
 - 2회차: $r = 100 + 10^2 = 200 \Rightarrow \Delta x = -\frac{1}{\sqrt{200}} \cdot 10 \approx -0.71$
 - 100회차: $r = 10,000 \Rightarrow \Delta x = -\frac{1}{\sqrt{10,000}} \cdot 10 = -0.1$

똑같은 기울기(10)가 나와도 시간이 갈수록 보폭이 눈에 띄게 줄어듭니다. 결국 너무 오래 학습하면 분모가 무한히 커져 보폭이 0에 수렴하는 **학습 정체** 현상이 발생합니다.

RMSProp

핵심 아이디어: "너무 먼 과거의 기록까지 더하지 말고, **지수 이동 평균**을 통해 최근의 지형만 반영하여 보폭을 유지하자."

- 수식: $r_k = \gamma r_{k-1} + (1 - \gamma)g_k^2$ (감쇠율 $\gamma = 0.9$ 가정)
- 예제 (매 스텝 기울기 $g = 10$ 이 일정하게 발생하는 경우):
 - 1회차: $r = 0.9(0) + 0.1(100) = 10 \Rightarrow \Delta x = -\frac{1}{\sqrt{10}} \cdot 10 \approx -3.16$
 - 2회차: $r = 0.9(10) + 0.1(100) = 19 \Rightarrow \Delta x = -\frac{1}{\sqrt{19}} \cdot 10 \approx -2.29$
 - 무한 반복 시: r 은 100으로 수렴 $\Rightarrow \Delta x = -\frac{1}{\sqrt{100}} \cdot 10 = -1.0$

AdaGrad와 달리 보폭이 일정 수준(-1.0) 이하로 떨어지지 않고 유지됩니다. 덕분에 장기적인 딥러닝 학습이 가능해집니다.

AdaDelta

핵심 아이디어: "학습률(η)을 설정하지 말고, 대신 과거에 실제로 이동했던 거리를 기준으로 보폭을 정하자."

- 핵심 메커니즘: $\Delta x \approx \frac{\text{RMS}(\Delta x_{\text{past}})}{\text{RMS}(g_{\text{now}})} \cdot g_{\text{now}}$

학습률 하이퍼파라미터 없이도 지형에 맞춰 스스로 움직이도록 합니다.

Adam

핵심 아이디어: "방향은 모멘텀(v , 관성)으로, 보폭은 RMSProp(r , 적응적 학습률)으로 조절하자."

- 예제: $\gamma_2 = 0.99$ 일 때 첫 번째 기울기 $g = 10$ 발생 시
 - 보정 전 r : $r_1 = (1 - 0.99) \cdot 10^2 = 1$ (값이 너무 작아 보폭이 될 위험이 있음)
 - 편향 보정 $\hat{r}$: $\hat{r}_1 = \frac{1}{1 - 0.99^1} = \frac{1}{0.01} = 100$
 - 최종 보폭: $-\frac{1}{\sqrt{100}} \cdot 10 = -1.0$

초기값이 0이라서 발생하는 통계적 왜곡을 **편향 보정(Bias Correction)**으로 해결합니다.

